

INF322
2023-2024



INF322 : Base de données SQL Procédural (Programmation SQL)

Juin 2024

Valéry MONTHE

valery.monthe@facsciences-uy1.cm

Université de Yaoundé 1





- Oracle possède son PL/SQL pour la programmation
- MS SQL server son Transact-SQL
- MySQL permet aussi de faire de la programmation

procédurale en SQL :

- ✓ les procédures stockées
- ✓ Les fonctions
- ✓ Les triggers



- **Procédures stockées**
- **Fonctions**
- **Trigger**



- ☐ Petit programme stocké dans la base de données
- ☐ Appelable à partir d'un client comme on peut le faire pour une requête.
- ☐ Est exécutée par le serveur de base de données.



❑ Délimiteur

- Il faut modifier le délimiteur par défaut de MySQL qui est le point virgule(;;).
- Les plus utiliser | et //

❑ Paramètres

- Définit par son sens, nom et son type
- 3 sens :
 - ✓ **IN** : entrant, valeur fournie à la procédure pour ses calculs
 - ✓ **OUT** : sortant, valeur produite par la procédure et pourra ensuite être utilisée en dehors de la procédure.
 - ✓ **INOUT** : entrant-sortant



☐ Syntaxe

DELIMITER |

CREATE PROCEDURE nom_proc ([param[,param...]])

BEGIN

instructions ;

END |

☐ Exécution

DELIMITER ;

CALL nom_proc(param);



❑ Exemple 1

DELIMITER |

CREATE PROCEDURE effectif_etudiants_regions()

BEGIN

SELECT codereg, **count(mat)**

FROM etudiant

GROUP BY codereg;

END |

DELIMITER ;



❑ Exemple 2 : paramètre entrant

DELIMITER |

```
CREATE PROCEDURE effectif_region(IN cod CHAR(2))
```

```
BEGIN
```

```
    SELECT codereg, count(mat)
```

```
    FROM etudiant
```

```
    WHERE codereg=cod
```

```
    GROUP BY codereg;
```

```
END |
```

```
DELIMITER ;
```




❑ Exemple 3 : paramètre de retour(sortant)

DELIMITER |

CREATE PROCEDURE effectif_regionParamOUT(OUT nb **INT**, IN
cod **CHAR**(2))

BEGIN

SELECT count(mat) INTO nb

FROM etudiant

WHERE codereg=cod

GROUP BY codereg;

END |

DELIMITER ;



❑ Exemple 3 : récupérer le résultat

```
SET @n=0;
```

```
CALL effectif_regionParamOUT(@n);
```

```
SELECT @n;
```



☐ Déclaration des variables

```
DECLARE myvar CHAR(10);  
DECLARE i, j INT DEFAULT 0;
```

☐ Commentaires

```
/*  
 *une ligne  
 *deuxieme ligne  
 */  
-- une seule ligne (il y a un espace après les 2 tirets)
```

☐ Affectation directe de variables

```
DECLARE x INT;  
DECLARE nom VARCHAR(50);  
SET x=10;  
SET nom='toto';
```



❑ Déclaration des variables : Exemples

```
DELIMITER |

CREATE PROCEDURE aujourd'hui_demain()
BEGIN
    DECLARE aujourd'hui DATE DEFAULT CURRENT_DATE();
    DECLARE demain DATE DEFAULT CURRENT_DATE();
    -- On déclare deux variables locales et on leur met une valeur par défaut

    SET demain = aujourd'hui + INTERVAL 1 DAY;
    -- On change la valeur de la variable locale
    SELECT Concat("Aujourd'hui c'est ", DATE_FORMAT(aujourd'hui, '%W %e %M %Y')) AS "Les jours"
    UNION
    SELECT Concat("Demain sera ", DATE_FORMAT(demain, '%W %e %M %Y')) AS "Les jours";
END |

DELIMITER ;
```



- ❑ S'exécute aussi sur le serveur.
- ❑ Retourne une valeur
- ❑ Peut être utilisée directement dans une requête SQL.

❑ Syntaxe :

```
CREATE FUNCTION name (params) RETURNS TypeRetour  
Begin  
    instruction;  
end
```



☐ IF THEN ELSE

IF var = 2 THEN ... ELSE ... ELSEIF ... END IF;

☐ CASE var

CASE var

WHEN 1 THEN ...;

WHEN 2 THEN ...;

ELSE ...; // autres cas

END CASE;

☐ REPEAT

REPEAT

...

UNTIL var = 5 END REPEAT;

☐ WHILE

WHILE i < 100 DO

...

END WHILE



- ❑ Va permettre de parcourir un jeu d'enregistrements.
- ❑ On déclare le curseur en l'associant à une requête de type SELECT qui va fournir les enregistrements.

❑ Syntaxe

```
DECLARE mycursor CURSOR FOR SELECT id, nom FROM matable;  
DECLARE var_id INT;  
DECLARE var_nom VARCHAR(50);  
OPEN mycursor;  
REPEAT  
    FETCH mycursor INTO var_id, var_nom;  
...  
  
    LEAVE boucle;  
UNTIL cond  
END REPEAT;  
CLOSE mycursor;
```



❑ Définition

- Procédure rattachée directement à un événement sur une table. INSERTION, SUPPRESSION, MISE A JOUR.
- s'exécute à un moment : AVANT ou APRES

❑ Syntaxe

```
CREATE TRIGGER <nomtrigger> <action_time> <event> ON  
<table>  
FOR EACH ROW  
BEGIN  
    instructions;  
END;
```




❏ Exemple

DELIMITER |

```
CREATE TRIGGER updatenotes AFTER UPDATE ON notes
FOR EACH ROW BEGIN
    DECLARE my_date date default '2019-01-10';
    DECLARE my_time time default '10:10:10';
    DECLARE my_user text;
    BEGIN
        set my_date = current_date;
        set my_user = current_user;
        set my_time= current_time;
        INSERT INTO `tracenotes2`(`IDUE`,`MATRICULE`,`CODEEVALUATION`,`IDSEMESTRE`,`ANNEE`,`ANONYMAT`,
                                `NOTE`,`oldIDUE`,`oldMATRICULE`,`oldCODEEVALUATION`,`oldIDSEMESTRE`,
                                `oldANNEE`,`oldANONYMAT`,`oldNOTE`,`agent`,`dateOp`,`timeOperation`,
                                `observation`)
        values (NEW.idue, NEW.MATRICULE, NEW.CODEEVALUATION, NEW.IDSEMESTRE, NEW.ANNEE,
                NEW.ANONYMAT,NEW.NOTE,OLD.idue, OLD.MATRICULE,
                OLD.CODEEVALUATION, OLD.IDSEMESTRE, OLD.ANNEE,
                OLD.ANONYMAT,OLD.NOTE,my_user,my_date, my_time,'NOTE MODIFIEE');
    END;
END;
|
DELIMITER ;
```